Valid Parentheses Demonstration

Title: Valid Parentheses using Java Collection Framework

Objective / Aim :

To write and execute a Java program to determine whether a string containing parentheses "()", curly braces "{}" and square brackets "[]" is valid using the Stack data structure.

System Requirements :

Hardware:

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage
- Keyboard and Monitor

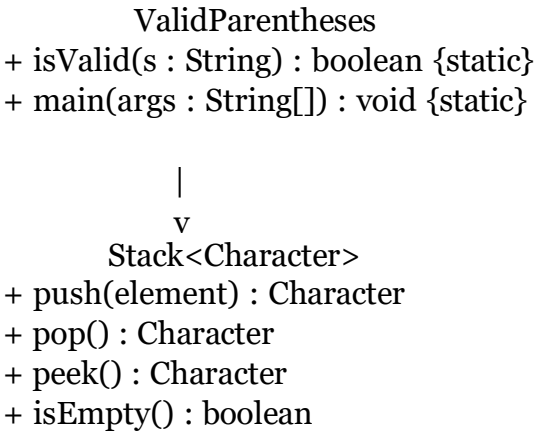
Software:

- Operating System: Windows
- Programming Language: Java
- Java Development Kit (JDK 17 or JDK 21)
- Visual Studio Code / Eclipse IDE or any Java IDE
- Command Prompt/Terminal

Algorithm / Logic Description :

1. Start the program.
2. Read a string containing brackets from the user.
3. Create an empty stack to store opening brackets.
4. Traverse each character of the string.
5. If the character is an opening bracket (, {, or [, push it onto the stack.
6. If the character is a closing bracket, check whether the stack is empty.
7. If the stack is empty or the top opening bracket does not match the closing bracket, return false.
8. Otherwise, pop the matching opening bracket from the stack.
9. After processing all characters, if the stack is empty, the string is valid; otherwise, it is invalid.
10. Display the result.
11. Stop the program.

UML Class Diagram :



Relationship: The ValidParentheses class utilizes the Stack class from the Java Collection Framework to manage and validate balanced bracket sequences.

Source Code :

File: ValidParentheses.java

```
import java.util.Scanner;
import java.util.Stack;

public class ValidParentheses {

    public static boolean isValid(String s) {
        Stack<Character> stack = new
        Stack<>();

        for (int i = 0; i < s.length(); i++) { char
            ch = s.charAt(i);

            // Push opening brackets onto the stack
            if (ch == '(' || ch == '{' || ch == '[') { stack.push(ch);
            } else {
                if (stack.isEmpty()) {
                    return false;
                }

                char top = stack.pop();

                // Check whether the brackets
                match if ((ch == ')' && top != '(') ||
                    (ch == '}' && top != '{') ||
                    (ch == ']' && top != '[')) {
                    return false;
                }
            }
        }
```

```
        return stack.isEmpty();
    }

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

        System.out.print("Enter the string: ");
        String s = sc.nextLine();

        if (isValid(s)) {
            System.out.println("Output:
            Valid");
        } else {
            System.out.println("Output: Not Valid");
        }

        sc.close();
    }
}
```

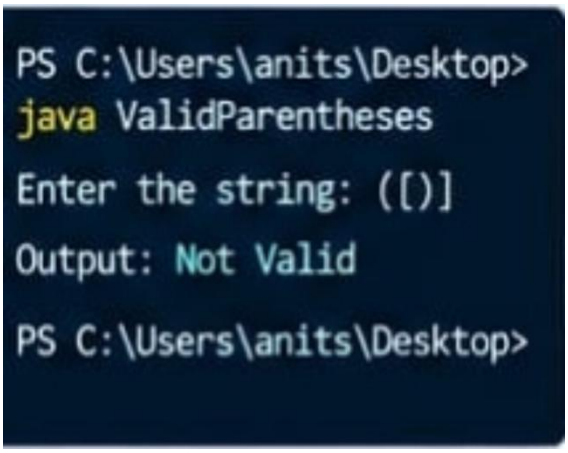
Compilation & Execution Commands :

Save file as: ValidParentheses.java

Compilation: javac ValidParentheses.java

Execution: java ValidParentheses

Outpu



Result :

Thus, the Java program to check whether a string contains valid parentheses was implemented and executed successfully using the Stack data structure from the java